

Vibe Coding for HVAC: Enabling AI-Enhanced Optimization in Building Automation Systems

Ben Bartling
Energy Engineer, Intellimation, LLC
ben@intellimation.net

Jayson Bursill, PhD, P.Eng,
Data Scientist, Delta Controls
jbursill@deltaccontrols.com

July 12, 2025

Abstract

The building automation industry remains constrained by proprietary, visual-flow programming environments that limit the development of advanced supervisory-level control logic. In contrast to modern software engineering, most commercial BAS platforms lack a native integrated development environment (IDE) supporting high-level, general-purpose programming languages. To bridge this gap, a methodology known as *vibe coding* is proposed: an AI-assisted development workflow for rapidly prototyping and deploying optimization algorithms. This approach is demonstrated through a rarely leveraged feature of the Tridium Niagara 4 platform: the Java-based Program Object. This component enables the implementation of computationally efficient yet advanced strategies, such as optimal start routines and trim-and-respond control, without relying on external libraries. These self-contained algorithms are well-suited to the capabilities of resource-constrained supervisory controllers. We argue that the future of BAS innovation hinges on manufacturers adopting standardized programming interfaces that support modern, C-style high-level languages. Doing so would empower engineers across disciplines within the BAS and Smart Building IoT sector to apply modern software engineering practices and deliver intelligent, energy-efficient control logic that next-generation buildings demand.

1 Introduction

Vibe coding introduces the use of modern computer science practices—such as lifecycle-driven programming, modular logic, and version-controlled development workflows—within supervisory-level building automation systems (BAS), coupled with AI-assisted code generation. Adapted from contemporary software development methodologies gaining traction in the broader tech industry [1], this approach leverages large language models (LLMs) to assist in generating supervisory control logic in high-level languages such as **Java**, **Python**, **C++**, **Lua**, or **C#**. These general-purpose languages, commonly taught in undergraduate computer science courses, are expressive enough to model complex control algorithms while remaining lightweight enough to execute on embedded supervisory controllers.

However, most BAS manufacturers do not expose internal APIs that support general-purpose, C-style high-level languages. Among major commercial platforms, only one is known to offer such a capability. The majority still rely on proprietary or visual flow-based programming environments that limit abstraction, modularity, and reusability—core principles of modern software engineering. As a result, engineers are often constrained in their ability to implement intelligent and maintainable control logic, despite working with hardware fully capable of supporting such designs.

This limitation stands in contrast to both the flex-

ible APIs commonly available in IoT platforms and the simplistic scripting languages still prevalent in the BAS industry—such as Siemens’ PPCL, Delta’s GCL, or Schneider Electric’s PlainEnglish. These legacy scripting tools, originally developed for HVAC sequencing and often reminiscent of early Visual Basic, lack support for structured data types and modern programming constructs—features essential for computer science coursework at the university level. Ultimately, these proprietary languages are ill-suited for expressing the increasingly sophisticated sequences of operation required in high-performance buildings, which often demand algorithmic, computer science-based approaches. Yet many of these sequences—such as the trim-and-respond routines defined by ASHRAE Guideline 36—can be implemented using simple mathematical operations, without the need for scientific computing or machine learning libraries, provided that clean, structured code is available.

One particularly compelling application of vibe coding—beyond typical HVAC optimization for office buildings, school districts, and other relatively straightforward systems—is in the development of computer science-driven algorithms for more complex central plant operations. While many commercial buildings are large enough to warrant a BAS, their systems are often simpler than those found in laboratories, hospitals, or mission-critical facilities. In contrast, central plants serving these more sophisticated environments demand advanced control strategies such as chiller sequencing, load balancing, and dynamic setpoint adjustment. These strategies can be effectively expressed using fundamental data structures and algorithms commonly taught in introductory computer science courses. When implemented using modern programming patterns, such algorithms become easier to maintain, extend, and debug—and they often outperform traditional flow-based implementations in both clarity and computational efficiency.

To support these emerging use cases, BAS manufacturers could evolve their supervisory-level platform controllers to provide high-level programming interfaces that are both expressive and computationally efficient. This would enable engineers to develop

and deploy robust, intelligent control strategies using familiar programming workflows—without relying on external infrastructure or heavyweight runtime environments.

2 Methodology

To implement advanced HVAC optimization strategies, this work moves beyond the limitations of conventional flow-based programming and restrictive proprietary scripting tools. While simple setpoint resets can be achieved using standard BAS logic blocks, more dynamic supervisory control sequences—such as trim and respond algorithms inspired by ASHRAE Guideline 36 [2]—benefit significantly from a programmatic approach. Our methodology focuses on developing a suite of reusable, AI-assisted control modules using the Java-based Program Object interface within the Tridium Niagara 4 platform.

The Niagara framework includes a special component known as the *Program Object*, which acts as a sandboxed scripting environment. By default, this object exposes three boilerplate lifecycle methods: `onStart()`, `onExecute()`, and `onStop()`. These methods are empty placeholders, as shown in Figure 1, but serve as entry points for executing custom control logic at different points in the object’s lifecycle.

The image shows a screenshot of the Niagara 4 software interface. At the top, there are two status indicators: a green circle labeled 'Program is up-to-date.' and an orange circle labeled 'Program is not signed.' Below these are tabs for 'Edit', 'Slots', 'Imports', and 'Source'. The 'Source' tab is active, displaying the following Java code:

```
public void onStart() throws Exception
{
    // start up code here
}

public void onExecute() throws Exception
{
    // execute code (set executeOnChange flag on inputs)
}

public void onStop() throws Exception
{
    // shutdown code here
}
```

Figure 1: Default Program Object in Niagara 4, exposing lifecycle methods `onStart()`, `onExecute()`, and `onStop()`.

The core of this methodology is a collaborative, human-on-the-loop workflow that synergizes the domain expertise of a controls engineer with the

code-generation capabilities of large language models (LLMs). The process unfolds in three stages:

1. **Strategy Definition:** The controls engineer, leveraging deep knowledge of HVAC systems, defines the objectives and operational parameters for a specific control strategy.
2. **AI-Assisted Prototyping:** Using LLMs such as Gemini or ChatGPT, the engineer provides structured prompts describing the desired control logic. The AI responds with Java code that follows a computer science-based, algorithmic approach, by default targeting the `onStart()`, `onExecute()`, and `onStop()` methods of the Program Object to accomplish the control engineer’s objectives.
3. **Integration and Validation:** The controls engineer copies the generated code into the Program Object editor within Niagara Workbench and compiles the Java logic using a menu option button. Once compiled, the Program Object becomes a flow-programming block with embedded algorithmic logic that can be placed onto the wire sheet alongside other components. The controls engineer then validates the HVAC sequence using standard Niagara wire sheet practices and interprets any compiler errors. These errors are fed back into the LLM to iteratively refine the code—repeating the process as needed until the desired objectives are achieved. This iterative, human-in-the-loop workflow reflects standard software engineering practices in the modern era of AI.

This workflow dramatically reduces development time and complexity. While traditional implementations might rely on numerous interconnected logic blocks, a single Program Object can encapsulate complex control logic in a clean and modular form—reducing clutter on the wire sheet and improving maintainability. Figures 2 and 3 (presented later in this paper) demonstrate finalized implementations of Program Objects generated using this methodology.

A prime example is the development of an AHU duct static pressure reset algorithm, shown in Figure 2. This module implements a simplified, field-ready version of trim-and-respond logic. Using the AI-assisted workflow, this complex algorithm was prototyped in hours—a task that would traditionally take days or even weeks of manual Java development.

Parameter	Value
Program	23.37 [ok]
vavDuctPos1	23.37 [ok]
vavDuctPos2	23.37 [ok]
vavDuctPos3	23.37 [ok]
vavDuctPos4	23.37 [ok]
vavDuctPos5	23.37 [ok]
vavDuctPos6	23.37 [ok]
vavDuctPos7	23.37 [ok]
vavDuctPos8	23.37 [ok]
vavDuctPos9	23.37 [ok]
vavDuctPos10	23.37 [ok]
vavDuctPos11	23.37 [ok]
vavDuctPos12	23.37 [ok]
vavDuctPos13	23.37 [ok]
vavDuctPos14	23.37 [ok]
vavDuctPos15	23.37 [ok]
vavDuctPos16	23.37 [ok]
vavDuctPos17	23.37 [ok]
vavDuctPos18	23.37 [ok]
vavDuctPos19	23.37 [ok]
vavDuctPos20	23.37 [ok]
vavDuctPos21	23.37 [ok]
vavDuctPos22	23.37 [ok]
vavDuctPos23	23.37 [ok]
vavDuctPos24	23.37 [ok]
vavDuctPos25	23.37 [ok]
vavDuctPos26	23.37 [ok]
vavDuctPos27	23.37 [ok]
vavDuctPos28	23.37 [ok]
vavDuctPos29	23.37 [ok]
ignoreCount	1.00 [ok] @ def
updateIntervalSeconds	300.00 [ok] @ def
ahuDuctStatus	true [ok] @ 30
ahuDuctPressStepOut	0.60 [ok]
trimInterval	0.10 [ok]
ductPressMin	0.50 [ok]
ductPressMax	1.00 [ok]
startUpDelaySeconds	120.00 [ok] @ def
vavDuctFilterMax	38.33 [ok]
vavDuctFilterMin	30.00 [ok]
vavDuctFilterMax	31.25 [ok]
statusTrace	Trim + Decrease SP + VAV Max = 38.33 + SP = 0.60 [ok]

Figure 2: AHU duct static pressure reset algorithm implemented in Niagara 4 using a Program Object.

Following this model, a comprehensive suite of reusable optimization blocks was developed. These blocks can be easily copied within the Niagara Workbench to other supervisory controllers—known as Java Application Control Engines (JACEs)—and can also be exported, zipped, and archived as `.bog` files for sharing outside the local Niagara environment. The developed blocks include:

- **AHU Duct Static Pressure Reset:** Reduces fan energy by dynamically lowering duct static pressure based on actual VAV damper demand.
- **AHU Supply Air Temperature Reset:** Adjusts supply air temperature to reduce reheat and cooling energy by responding to zone demand and outdoor air conditions.
- **Predictive Chiller Enable/Disable Logic:** Avoids unnecessary chiller operation and prevents short cycling based on aggregated building load.

- **Advanced Chiller Rotation:** Ensures equitable runtime distribution across chillers for improved reliability and lifecycle management.
- **Optimal Start Control:** Reduces pre-occupancy energy use by learning the building’s thermal response and starting HVAC systems only as early as needed.
- **Custom Reactive and Delay Components:** Provide fine-grained timing logic for stable, event-driven supervisory control.

3 Results

The application of vibe coding yielded tangible results, demonstrating that large language models (LLMs) can assist in the rapid development of HVAC control algorithms suitable for real-world deployment. In practice, the generated logic operated reliably on supervisory hardware, and when integrated with IoT platforms, telemetry was captured and visualized through tools like Grafana to support diagnostics, validation, and continuous commissioning.

Among the algorithms explored, the optimal start routine emerged as both the most technically involved and the most impactful. This strategy is supported by research from the Pacific Northwest National Laboratory (PNNL), which found that optimal start algorithms can reduce electricity consumption by 20–25% in small commercial buildings [3]. Our implementation focused on a simplified, computationally efficient version of this approach—referred to in the research as Model 1—which uses an exponentially weighted moving average (EWMA) to learn the optimal lead time for starting HVAC equipment based on historical system performance. The algorithm is self-tuning and adapts to changing conditions over time.

The mathematical operations used in our Java implementation—readily transferable to any general-purpose C-style language—are intentionally kept simple to ensure compatibility with resource-constrained supervisory controllers. The control logic primarily involves basic arithmetic operations (addition, subtraction, multiplication, division) and ele-

mentary statistical functions such as averaging and min/max, all implemented without reliance on external libraries. Even the most complex routine—the optimal start algorithm—uses an EWMA formula coded entirely from scratch in Java.

While this work emphasizes lightweight constructs typically found in introductory data structures and algorithms courses, the methodology is extensible to more advanced strategies. Techniques such as priority queues for scheduling, finite-state machines for mode transitions, and basic regression models for prediction can all be implemented if desired. These are typically developed manually using only native language features, without reliance on scientific computing or machine learning libraries, to ensure the resulting logic remains computationally efficient, deterministic, and well-suited for deployment on embedded supervisory platforms.

However, some manufacturers could potentially support limited scientific computing libraries, particularly if hardware resources allow. In such cases, controls engineers and technicians would need to manage and monitor system resource usage closely—an existing best practice in BAS contracting environments. Manufacturers already train technicians to track the performance and capacity of supervisory-level devices, so extending this practice to include resource management for higher-level computation would be a natural evolution.

Despite these limitations, the deliberate use of clean, modular algorithms grounded in core computer science principles enables the creation of intelligent, maintainable, and energy-efficient control strategies. When thoughtfully applied, these building blocks can support sophisticated supervisory-level optimization that operates within the real-time constraints of modern building automation systems.

In this case, the LLM successfully generated the core Java logic for the optimal start algorithm using structured prompts. A controls engineer then reviewed, refined, and integrated the logic within Niagara Workbench. A completed implementation is shown in Figure 3. The Program Object receives key inputs such as the occupancy schedule, zone temperature, and setpoint. It calculates the required equipment start time and asserts a `true` output to over-

ride the default schedule when needed. The object also outputs trendable diagnostic metrics—including start-time temperatures and heating or cooling rate estimates—which are fed back into the EWMA model to enable ongoing self-adjustment.

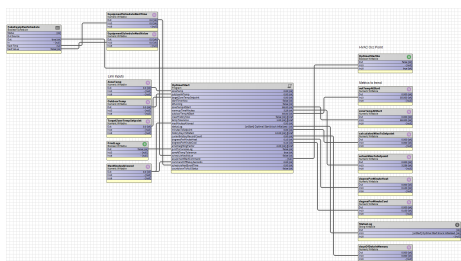


Figure 3: Self-tuning optimal start algorithm.

4 Discussion and Future Work

The success of the **vibe coding** methodology suggests a clear path forward for the building automation industry: the adoption of standardized, lifecycle-driven programming interfaces that support modern development workflows. One example is the Tridium Niagara 4 Program Object, which enables high-level logic implementation using general-purpose languages. Similarly, platforms such as Delta Controls’ enteliWEB environment offer Python-based scripting features for supervisory-level control. Future research could explore the capabilities, limitations, and design patterns supported by these and other interfaces, helping to define common requirements and inform the development of interoperable, efficient, and secure programming models across the industry. By evaluating multiple implementations in an objective, comparative framework, researchers can help guide manufacturers and practitioners toward solutions that promote maintainability, innovation, and energy efficiency.

Standardizing these interfaces around commonly taught, general-purpose high-level programming languages—such as those used in undergraduate computer science curricula—is critical for managing the growing complexity of modern energy systems. As

technologies like grid-interactive efficient buildings (GEBs), demand response, and advanced chiller plants become more prevalent, **HVAC mechanical engineers** are tasked with designing increasingly sophisticated sequences of operation. A modern programming environment enables **controls engineers and technicians** to implement this design intent with greater precision, scalability, and maintainability than traditional flow-based programming tools typically allow.

However, three significant hurdles remain: proprietary platforms, the workforce skills gap, and cybersecurity.

1. **Proprietary Platforms:** The BAS industry remains highly proprietary, with manufacturer-specific APIs and tools that are unfamiliar not only to large language models (LLMs), but also to graduates of modern computer science, IT, industrial technology, and engineering programs. Just as LLMs lack native understanding of these closed ecosystems, so too do many early-career professionals entering the BAS field. To help bridge this gap, we developed a “context engineering” document—a detailed `README.md` file that can be provided to the LLM as part of the prompt, supplying the necessary context to generate more accurate and relevant control logic code [4]. This technique offers a repeatable and adaptable method for aligning general-purpose AI tools with the specialized, proprietary environments common in BAS development.
2. **Workforce Skills Gap:** The skill set required of a **BAS field technician** is already extensive and highly multidisciplinary. These professionals must be experts in mechanical system sequencing, low-voltage control circuitry wiring, electronics troubleshooting, HVAC test and balance procedures, and often PLC or proprietary BAS programming. Introducing computer science into this already demanding role may seem daunting. However, vibe coding aims to lower this barrier by requiring only a foundational understanding of algorithms and control logic—comparable to the level covered in introductory computer science courses. With this

baseline, technicians can effectively collaborate with AI tools, guide large language models using domain-specific prompts, and validate the outputs to ensure safe, reliable, and efficient execution of complex control sequences specified by design engineers. Rather than replacing traditional skills, vibe coding complements them by enabling a more scalable, modular, and intelligent approach to programming supervisory-level BAS logic.

3. **Cybersecurity Risks:** Enabling general-purpose programming on supervisory controllers introduces significant cybersecurity considerations. To mitigate these risks, any such implementation must adopt a sandboxed model—a controlled execution environment where the code is tightly restricted in what it can access or affect. In this context, sandboxing means the high-level language is confined to a predefined set of safe operations and is isolated from sensitive system functions. For example, Niagara’s Program Object is sandboxed in that it can only interact with the platform’s internal API, and it cannot call external libraries, access the operating system directly, or initiate arbitrary network communications.

This level of isolation is critical. Without sandboxing, a compromised controller could be exploited to traverse from the operations technology (OT) network into the information technology (IT) network, posing a serious security threat—especially since supervisory controllers are often VPN-connected to external vendors for remote maintenance. A sandboxed programming model ensures that even if logic is written in a general-purpose language, it remains constrained within the manufacturer’s framework, effectively preventing lateral movement, data exfiltration, or the misuse of protocol stacks. This is essential for maintaining security while still enabling flexible, modern development workflows within building automation systems.

Looking ahead, the potential for AI in building automation extends beyond text-based code generation.

A compelling future direction would be to integrate LLMs more deeply into BAS development environments like Niagara Workbench. A native chat interface powered by a context-aware AI agent could help technicians generate, edit, and deploy logic blocks seamlessly, further closing the gap between complex design and practical field implementation.

To enable this broader vision, BAS manufacturers should consider adopting a standardized model similar to the Niagara Program Object—one that supports a sandboxed, high-level programming interface using a general-purpose C-style language. Such interfaces would allow engineers to apply contemporary software engineering patterns natively within supervisory controllers while maintaining strong cybersecurity boundaries. Importantly, the specific language need not be identical across platforms: whether a vendor chooses Java, Python, or another general purpose high level C-based language, LLMs can assist in translating between them with ease and accuracy, especially when the control algorithms remain relatively simple from a computational standpoint.

This architecture acknowledges the dual role of the modern BAS: supervisory controllers are responsible for executing core control logic, while IoT infrastructure supplements the system with advanced capabilities such as machine learning models, data extraction pipelines, and cloud-based analytics. The goal is to keep the core logic lightweight—enabling local contractors to easily configure and commission HVAC automation hardware—while still being powerful enough to support complex control sequences built on modern computer science patterns.

Ultimately, these efforts are being led by the research community, the smart building and IoT sectors, and the building optimization industry—with potential participation from manufacturers if they choose to adopt these practices—and can lay the groundwork for broader industry standards such as ASHRAE’s Control Description Language (CDL), currently being formalized as Standard 231P. By applying modern software engineering principles to supervisory control in BAS and IoT contexts, this work demonstrates the practicality and advantages of using general-purpose, high-level languages for implementing building control logic.

As manufacturers begin to align with these practices and provide programmable interfaces that reflect contemporary development workflows, the insights gained through approaches like vibe coding can directly inform the structure, tooling, and real-world applicability of CDL. Importantly, the algorithms used in these control sequences—when studied and published in open-source academic environments—can be easily translated by AI across any C-style high-level language, dramatically lowering the barrier to innovation. This promotes collaboration between academia, industry, and open-source communities, enabling rapid prototyping and knowledge sharing across platforms.

Breaking down the proprietary barriers that have historically dominated the BAS industry is essential to fostering this evolution. Doing so will not only accelerate technological advancement but will also contribute to the development of more robust, adaptable, and energy-efficient buildings—an urgent imperative in the face of climate change. If and when manufacturers adopt general-purpose languages across their platforms, those environments could serve as natural hosts for CDL-based logic. In the long term, this convergence has the potential to establish CDL as the universal control logic standard across the entire building automation industry.

References

- [1] Wikipedia contributors, *Vibe coding* — *Wikipedia, the free encyclopedia*, Accessed: July 4, 2025, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Vibe_coding.
- [2] ASHRAE, *Guideline 36: High-performance sequences of operation for hvac systems*, Available from ASHRAE bookstore, 2018.
- [3] Pacific Northwest National Laboratory, “Implementation and validation of optimal start control strategy for air conditioners and heat pumps,” Pacific Northwest National Laboratory, Tech. Rep. PNNL-34304, 2023. [Online]. Available: <https://www.pnnl.gov/publications/implementation-and-validation-optimal-start-control-strategy-air-conditioners-and-heat-pumps>.

`validation - optimal - start - control - strategy-air-conditioners-and-heat`.

- [4] B. Bartling, *Niagara vibe code addict github repository*, Accessed: July 4, 2025, 2025. [Online]. Available: <https://github.com/bbartling/niagara4-vibe-code-addict>.